TECHNISCHE UNIVERSITÄT DRESDEN

FACULTY OF COMPUTER SCIENCE
INSTITUTE OF SYSTEMS ARCHITECTURE
CHAIR OF DATABASES
PROF. DR.-ING. WOLFGANG LEHNER

Diplomarbeit

zur Erlangung des akademischen Grades
Diplom-Informatiker

Active Cache-Aware Hardware Adaptation: Cache Engine for Trie-Based Index Structures

Benjamin-Elias Probst
(Born 11. April 1996 in Potsdam)

Professor: apl. Prof. Dr.-Ing. Dirk Habich
Tutor: apl. Prof. Dr.-Ing. Dirk Habich, Dr.-Ing. Alexander Krause

Dresden, June 1, 2026

Hier Aufgabenstellung einfügen!

Selbstständigkeitserklärung

Hiermit erkläre ich, Benjamin-Elias Probst (Mat.-Nr. 4510512, geboren am 11. April 1996 in Potsdam), dass ich die von mir am heutigen Tag dem Prüfungsausschuss der Fakultät Informatik eingereichte Diplomarbeit zum Thema:

Active Cache-Aware Hardware Adaptation:
Cache Engine for Trie-Based Index Structures

vollkommen selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt sowie Zitate kenntlich gemacht habe.

Dresden, den 1. Juni 2026

Benjamin-Elias Probst
Mat.-Nr. 4510512

Kurzfassung
Platzhalter – deutsche Kurzfassung folgt.

Abstract
Placeholder – English abstract to follow.

Contents

1 Introduction

1.1 Motivation

Trie-based index structures are the backbone of modern in-memory databases. Since the Adaptive Radix Tree (ART) by Leis et al. [**?**] in 2013, research has diversified in several directions: trie-height optimization (HOT [**?**]), hybrid B^+ structures (Masstree [**?**], B^2-tree [**?**]), succinct representations (CoCo-Trie [**?**], SuRF [**?**]), and self-tuning approaches (START [**?**]).
In parallel, a dedicated body of research on cache awareness, prefetching, and allocation strategies has emerged. Heuristics such as Cache-Sensitive Search Trees, cache-oblivious layouts, and hierarchical prefetching each address an individual axis of the cache hierarchy — yet they have never been systematically combined within a single, permuted measurement architecture.
This thesis closes that gap with a three-layer cache-engine architecture (CacheEngine $\rightarrow$ ExecutionEngine $\rightarrow$ SearchEngine) whose building blocks are decomposed into fourteen orthogonal permutation axes, spanning a configuration space of more than 10^{11} permutations. As a probe for state-of-the-art membership we contribute PRT-ART (a Probst Redirect Tree / ART hybrid) with eight building-block layers of its own.

1.2 Problem Statement

Existing cache-aware index designs optimize one or a few aspects in isolation and report results that are hard to compare across publications, because each work uses its own data structures, workloads, and measurement methodology. There is no common framework that (i) decomposes such designs into interchangeable axes, (ii) reconstructs published algorithms as explicit configurations, and (iii) measures both individual axes and whole algorithms on a single, fair basis.

1.3 Research Questions

RQ1 (Composition) Which algorithm axes of a cache-aware trie index can be canonically systematized in a permutation matrix, such that state-of-the-art implementations can be reconstructed as profile configurations?

RQ2 (Measurement methodology) How can a fair, reproducible YCSB workload run be performed per permutation, allowing profile-specific workload selection (e.g. a range-scan profile $\rightarrow$ YCSB-E) while minimizing the measurement overhead via an opt-in experiment mode?

RQ3 (Probe comparison) How does PRT-ART perform as a probe against the eight Tier-1 SOTA designs (ART, HOT, Masstree, CoCo-Trie, START, B^2-tree, Wormhole, SuRF) and the Tier-2/3 SOTA under the three mandatory measurement series A (probe vs. SOTA), B (SOTA-only baseline), and C (merge points)?

1.4 Objectives and Contributions

The central contributions of this thesis are:

- A three-layer architecture (CacheEngine → ExecutionEngine → SearchEngine) with ABI-stable C++23 module interfaces and variadically templated hybrid-API specializations (1 parameter ⇒ `std::vector` API; 2 parameters ⇒ `std::map` API; $N > 2$ ⇒ `std::map<K, std::tuple<`$V_1 \ldots V_N$`>>`).
- A fourteen-axis permutation matrix with `std::variant` building blocks and compile-time fallback between the probe stack and the state-of-the-art stack.
- Thirty SOTA algorithm profiles as persisted XML configurations (8 Tier-1 + 22 Tier-2/3) that the `ExperimentDriver` transforms automatically into pre-compiled C++23 modules.
- PRT-ART as a probe with eight building-block layers of its own (4+2 pool allocator, optimistic lock coupling with reserved blocks, multi-level layout, path-oriented prefetch, density tracker, hypothesis metrics H1/H2/H3, inline/external/chain-ref value handles, signaling-bit serialization).
- A measurement driver with defined-/full-mode distinction and profile-aware workload routing.

1.5 Structure of the Thesis

Chapter **??** introduces the necessary background. Chapter **??** summarizes the state of the art across six search clusters (A–F) and an allocator cluster. Chapter **??** presents the axis library framework, the layered architecture, the ABI interface, and the fourteen-axis permutation matrix. Chapter **??** documents the implementation across the three repositories. Chapter **??** describes the measurement methodology with the three mandatory series and profile-aware workload routing. Chapter **??** presents and discusses the results, including the hypotheses H1–H3. Chapter **??** concludes with a summary and an outlook.

2 Fundamentals

This chapter establishes the background needed for the remainder of the thesis: the cache hierarchy and the notion of cache awareness, trie-based index structures with the Adaptive Radix Tree as the running reference, the unified `std::map` comparison interface, the YCSB workloads, and the C++23 metaprogramming facilities on which the cache engine is built.

2.1 Cache Hierarchy and Cache Awareness

Modern processors interpose several levels of cache (L1, L2, L3) between the registers and main memory. Data is transferred between these levels in fixed units called *cache lines*, typically 64 bytes. An access that finds its datum in a cache level is cheap; a miss propagates to the next, slower level, and a miss in the last-level cache reaches main memory at a cost of hundreds of cycles. Address translation adds a second hierarchy: the *translation lookaside buffer* (TLB) caches virtual-to-physical mappings, and a dTLB miss is itself expensive. On multi-socket systems, non-uniform memory access (NUMA) makes the latency of a reference depend on which node owns the page.
A data structure is *cache-aware* when its layout is tuned to concrete cache parameters (line size, level capacities), and *cache-oblivious* when it achieves good locality across all levels without knowing them. Both philosophies recur throughout the state of the art (Chapter **??**) and motivate the layout, prefetch, and hardware axes of the proposed engine.

2.2 Trie-Based Index Structures

A *trie* (digital/radix tree) indexes keys by their byte sequence rather than by comparison: each level discriminates on one or more key bytes, so the search-path length depends on key length, not on the number of stored keys. The *fanout* (branching factor) and the *node representation* dominate both space and cache behaviour. The Adaptive Radix Tree (ART) [**?**] adapts the node representation to the actual fanout, using four node types (Node4, Node16, Node48, Node256), and applies path compression (lazy expansion, prefix folding) to keep the tree shallow. ART serves as the running reference design throughout this thesis.

2.3 The Unified Comparison Interface (`std::map`)

To compare structurally different search algorithms fairly, they are all mapped onto a single, well-understood interface: the C++ ordered-map contract `std::map<Key, Value>` (`find`/`insert`/`erase`/range iteration). The axes introduced in Chapter **??** describe the *internal* behaviour behind this interface, while the interface itself stays fixed — comparing two conforming `std::map` implementations is therefore the central measurement act. A variadic specialization keeps the interface ergonomic: one template

parameter exposes a `std::vector`-like API, two parameters the `std::map` API, and $N > 2$ a `std::map<K, std::tuple<...>>`.

2.4 YCSB Workloads

The Yahoo! Cloud Serving Benchmark (YCSB) defines a family of key–value workloads used here as the common load profile: YCSB-A (50/50 read/update, Zipfian), YCSB-B (95/5 read/update), YCSB-C (read-only), YCSB-D (read-latest), YCSB-E (short range scans), and YCSB-F (read–modify–write). Each algorithm profile declares an expected workload (Chapter **??**), which the measurement driver uses for profile-aware routing (Chapter **??**).

2.5 C++23 Metaprogramming

The engine relies on compile-time composition to avoid run-time dispatch in the hot path. *Concepts* constrain the building-block interfaces; the *curiously recurring template pattern* (CRTP) provides static polymorphism; `if constexpr` and `std::conditional_t` select between a probe implementation and a cache-engine default per axis; and C++23 *modules* provide an ABI-stable boundary between the permutation binaries and the builder. Together these facilities realize the zero-cost abstraction on which the permutation matrix depends.

3 State of the Art

The state of the art in cache-aware trie index structures falls into six thematic search clusters (A–F) and a parallel allocator corpus (A01–A23). In total, 33 search-algorithm papers and 21 allocation papers were studied in depth; the results are available as persisted XML configurations under `algorithm_profiles/` in the `comdare-cache-engine`. The complete building-block and allocator matrices are given in Appendix **??**.

3.1 Trie Family (Cluster A)

P01 ART [**?**] introduces the Adaptive Radix Tree with adaptive node sizes Node4/16/48/256. **P02 HOT** [**?**] optimizes trie height via multi-byte prefixes and bit-vector indexes. **P03 Masstree** [**?**] combines a trie with B-trees and a per-internal-node permutation index. **P04 CoCo-Trie** [**?**] uses succinct-encoded compact pages. **P05 START** [**?**] extends ART with self-tuning of the node representation. P09 LOUDS (Jacobson 1989) is the succinct-trie foundation, and **P10 SuRF** [**?**] uses LOUDS for range filtering.

3.2 Hybrid and B⁺ Family (Cluster B)

P06 B^2-tree [**?**] employs a two-level fanout for combined L1+L2 cache awareness. **P07 Wormhole** [**?**] combines a hash table with a linear prefix view. P11 CSS-tree (Rao/Ross 1999) is the first cache-sensitive search tree; P12 CSB$^+$-tree (Rao/Ross 2000) develops it into a B$^+$-tree; P13 Hankins/Patel (2003) studies the effect of node size on cache performance.

3.3 Layout Theory (Cluster C)

P14 Samuel/Pedersen/Bonnet (2005) makes the CSB$^+$-tree processor-conscious. P15 Graefe/Larson (2001) studies B-tree indexes versus CPU caches. P16 Bender/Demaine/Farach-Colton (2002) introduces cache-oblivious tree layout (van Emde Boas); P17 Bender et al. (2005) extends it to cache-oblivious B-trees. P18 Saikkonen/Soisalon-Soininen (2008) and P19 (2016) develop multi-level and layout-invariant B-trees.

3.4 Prefetching (Clusters D and E)

P20 “B-Trees Are Back” (Mueller/Benson/Leis 2025) demonstrates modern B-tree engineering. P21 Chen/Gibbons/Mowry (2001) introduces prefetching B$^+$-trees; P22 Chen et al. (2002) develops fractal prefetching. P23 Khan (2010) makes cache prefetching dynamically adaptive; P24 Naderan-Tahan/Sarbazi-Azad (2016) adds an adaptive filter on the cache prefetcher. P25 Mahling/Weisgut/Rabl (2025) combines range scans with hot-path caching. P26 Zhang (FGCS 2024) and P27 Zhang (ASPLOS 2025, hierarchical

prefetching) are the most recent schemes, and P28 Kuehn (DaMoN 2023) motivates data-driven cache optimization.

3.5 Synchronization (Cluster F)

P08 “ART of Practical Synchronization” (Leis 2016) introduces optimistic lock coupling (OLC) for ART. P29 RCU (McKenney, OLS 2001) and P30 Hazard Pointers (Michael 2004) are the canonical reclamation schemes. P31 Ungethuem/Habich (2017) and P32 Schmidt/Habich (2025) are TU-Dresden-specific hardware-optimization surveys; P33 (VAMPIR poster, Berthold/Habich 2023) provides the OTF2 profiler integration.

3.6 Allocator Research (A01–A23)

The 21 allocation papers are catalogued systematically in the allocator matrix (Appendix **??**). Tier-1 representatives are Hoard (A01, ASPLOS 2000), Slab (A02, USENIX 1994), Michael lock-free (A03, PLDI 2004), mimalloc (A04, MSR-TR-2019-18), jemalloc (A05), TCMalloc (A06), snmalloc (A07, ISMM 2019), and NUMAlloc (A09, ISMM 2023). Tier-2/3 add CAMA (A12, RTAS 2011), StarMalloc (A13, OOPSLA 2024), Crystalline reclamation (A17, PLDI 2024), and PIM-malloc (A16, arXiv 2025).

3.7 Profile Schema with the `<expected_workload>` Tag

For each SOTA algorithm and each allocator, a profile XML in `cache_engine/algorithm_profiles/` records metadata, the axis assignment (page, node, traversal, value_handle, concurrency, allocator, prefetch, telemetry, isa, layout, reclamation), a key–value signature, and an optional `<expected_workload>` tag that overrides the `ExperimentDriver` heuristic (Chapter **??**). All 30 SOTA profiles and all 10 allocator profiles are tagged.
The workload distribution is 13× YCSB_C (read-heavy), 9× YCSB_A (Zipfian read+update), 6× YCSB_E (range scans), and 2× YCSB_B (95/5 read/update). PRT-ART itself is tagged YCSB_F (read–modify–write, for density-tracker updates). Table **??** lists the ten implemented allocator profiles.

3.8 Hardware and Scheduling Strategy per Paper

After the N-phase extension of the building-block matrix (Section **??**), two additional algorithm permutation axes are anchored:

- **Axis 12 Hardware-Strategy**: which hardware features the algorithm actively uses — sub-axes 12.1 SIMD family (AVX2/AVX-512/NEON/SVE2/scalar), 12.2 cache-level targeting (L1/L2/L3/HBM-aware), 12.3 NUMA strategy, 12.4 prefetch hardware (PREFETCH/PREFETCHNTA/PREFETCHW), 12.5 atomic-instruction family (CAS/LL-SC/RMW-extended).

- **Axis 13 Scheduling-Strategy**: how work is distributed — sub-axes 13.1 worker-pool layout (thread-per-core / work-stealing / CPU-pinning), 13.2 SIMD-worker-count limit (hardware-limited to

Table 3.1: SOTA algorithm profiles with workload affinity

Profile	**Paper ref.**	**expected_workload**
art	P01 (Leis 2013)	YCSB_C
hot	P02 (Binna 2018)	YCSB_C
masstree	P03 (Mao 2012)	YCSB_A
coco_trie	P04 (Boffa 2024)	YCSB_C
start	P05 (Fent 2020)	YCSB_C
b2tree	P06 (Schmeisser 2022)	YCSB_E
wormhole	P07 (Wu 2019)	YCSB_A
surf	P10 (Zhang 2018)	YCSB_A
css_tree	P11 (Rao/Ross 1999)	YCSB_C
csb_tree	P12 (Rao/Ross 2000)	YCSB_E
hankins	P13 (Hankins 2003)	YCSB_C
samuel	P14 (Samuel 2005)	YCSB_C
graefe	P15 (Graefe 2001)	YCSB_C
bender_treelayout	P16 (Bender 2002)	YCSB_C
bender_cacheoblivious	P17 (Bender 2005)	YCSB_C
saikkonen_multilevel	P18 (Saikkonen 2008)	YCSB_E
saikkonen_layoutinvariant	P19 (Saikkonen 2016)	YCSB_E
btreesareback	P20 (Mueller 2025)	YCSB_E
chen_prefetch	P21 (Chen 2001)	YCSB_A
chen_fractal	P22 (Chen 2002)	YCSB_A
khan_adaptive	P23 (Khan 2010)	YCSB_A
naderan_tahan	P24 (Naderan-Tahan 2016)	YCSB_A
mahling	P25 (Mahling 2025)	YCSB_E
zhang_fgcs	P26 (Zhang 2024)	YCSB_A
zhang_asplos	P27 (Zhang 2025)	YCSB_A
kuehn	P28 (Kuehn 2023)	YCSB_C
rcu	P29 (McKenney 2001)	YCSB_B
hazard_pointers	P30 (Michael 2004)	YCSB_B
ungethuem	P31 (Ungethuem 2017)	YCSB_C
to_stride	P32 (Schmidt 2025)	YCSB_C

Table 3.2: Allocator profiles with license and workload affinity

Profile	**Family**	**License**	**expected_workload**
hoard	A01	Apache-2.0	YCSB_A
michael_lockfree	A03	BSD-3	YCSB_B
mimalloc	A04	MIT	YCSB_A
jemalloc	A05	BSD-2	YCSB_B
tcmalloc	A06	BSD-3	YCSB_C
snmalloc	A07	MIT	YCSB_A
scalloc	A08	BSD-3	YCSB_A
rpmalloc	A10	Public Domain	YCSB_C
lrmalloc	A11	BSD-3	YCSB_B
dlmalloc	A20	CC0	YCSB_C

typically 2 of N cores), 13.3 heterogeneous-core dispatch (Intel hybrid P-/E-cores), 13.4 co-routine strategy, 13.5 batch granularity.

Table **??** shows a selection of these two axes per SOTA paper; the full catalogue is in Appendix **??**.

Table 3.3: Hardware and scheduling strategy per SOTA paper (selection)

Paper	**Hardware strategy (axis 12)**	**Scheduling strategy (axis 13)**
P01 ART	AVX2, L1-aware, none, CAS	thread-per-core
P02 HOT	AVX2 (bit-vector), L1-aware, CAS	thread-per-core
P03 Masstree	scalar, L1-aware, CAS	thread-per-core + work-stealin
P05 START	AVX2 (multi-byte span), L1-aware, CAS	thread-per-core
P20 LeanStore	AVX2, HBM-aware, NUMA-aware, CAS	work-stealing + hybrid
P27 hp-soft	PREFETCH (L1/L2/L3 bundle), all-cache, CAS	thread-per-core + co-routine
P28 Kuehn	PREFETCH (scalar counter), L1-aware, CAS	thread-per-core + sampling
P31 Ungethuem	AVX-512 (HBM), HBM-aware, NUMA-aware, CAS	hybrid (P-/E-cores)
P32 Schmidt	AVX-512 (SIMD stride), HBM-aware, NUMA-aware, CAS	hybrid
PRT-ART	**AVX2, L1-aware (TLB-aligned), local NUMA, PREFETCH, CAS (OLC)**	**thread-per-core + 2-SIMD-w**

3.9 Research Gap

Each of the surveyed works optimizes one or a few axes of the cache hierarchy in isolation, and the results are reported in incomparable settings. What is missing — and what this thesis contributes — is a systematic, orthogonal *axis decomposition* of these designs together with a permuted measurement architecture that reconstructs them as explicit configurations and measures them on one common basis.

4 Concept and Architecture

This chapter presents the core contribution of the thesis: an axis library framework that decomposes cache-aware search algorithms into interchangeable, orthogonal building-block axes, the layered engine architecture that realizes it, the ABI-stable module interface, the fourteen-axis permutation matrix, the probe PRT-ART, and the builder that orchestrates the three-stage evaluation.

4.1 The Axis Library Framework

The central idea is to treat an algorithm not as a monolith but as a *composition of orthogonal axes*, each axis representing one sub-decision of a cache-aware index (how pages are laid out, how nodes branch, how memory is allocated, how concurrency is handled, and so on). A concrete algorithm is then a point in the Cartesian product of all axes, and a state-of-the-art design is reconstructed as one explicit configuration. The decisive property is *modular interchangeability*: a researcher can study one new axis variant in isolation — and measure its effect, per axis and for the whole algorithm — without re-implementing all the others. This turns the comparison of two `std::map` implementations (Section **??**) into a controlled, reproducible experiment.

4.2 Four-Subsystem Separation (M Model)

The diploma project, the cache-engine builder, the cache engine, and the probe are *four formally separated subsystems* with distinct responsibilities. The separation is orthogonal to the three-repository split (Section **??**): builder and engine live in the same repository but are an executable and a library, respectively.

1. **`messung_driver`** (Diplomarbeit/Code) — the evaluation orchestrator (outer loop): reads the three measurement-series XML configurations (A/B/C), triggers the builder per series, and collects the results for the appendix.

2. **CacheEngineBuilder** (`cache-engine/apps`) — an autonomous platform-probing system implementing the seven-phase pipeline (discover, measure, classify, publish, bind, execute, compare) and enumerating the permutation space per registered engine.

3. **CacheEngine** (`cache-engine/libs`) — the tool library: twelve internal sub-engines C_1–C_{12} (layout, pinning, prefetch, coherence, telemetry, allocation, migration, encoding, heuristics, topology, scheduler, filter) and the cache-strategy families, exposed as a C++23 concept API.

4. **Probe (PRT-ART)** (`prt-art`) — an algorithm implementation registered with the cache engine as an `IExecutingEngine`; it implements the search-engine layer for every axis and consumes cache-engine services during its own lookup/insert/scan operations.

The defining property of the M model is the *bidirectional* cache-engine ↔ probe relationship: (a) the cache engine instantiates the probe per permutation configuration (builder phase 5, BIND), and (b) the probe calls cache-engine services (telemetry, prefetch, heuristics) during its own lookups at run time (phase 6, EXECUTE). This symmetry enables the *multi-probe* capability: measurement series A compares PRT-ART against many SOTA adapters in a single builder pipeline.

4.3 Three-Layer Hierarchy

```
CacheEngine          (layer 1: state of the art, tool library)
    ^  inherits
ExecutionEngine<ProcessingStrategy>   (layer 2: OS primitives)
    ^  inherits
SearchEngine<Collection, Config>      (layer 3: search patterns)
    ^  specializes
PrtArtSearchEngineAdapter             (probe binding)
```

Layer 1 (CacheEngine) provides cache-page topology, allocator families, sub-engines, and telemetry strategies as a library. **Layer 2 (ExecutionEngine)** wraps these into higher OS primitives (cache-line-aligned allocate, NUMA-conforming read-pin, coherence-aware write). **Layer 3 (SearchEngine)** offers search-specific patterns (trie walk, B^+ range scan, hot-path lookup) on top of the execution primitives.

4.4 ABI-Stable C++23 Module Interface

The ABI follows the variadic-template principle: one parameter ⇒ `std::vector` API, two parameters ⇒ `std::map` API, $N > 2$ ⇒ `std::map<K, std::tuple<`$V_1 \ldots V_N$`>>`. Complex keys are reduced to fixed-length 16-byte binary strings by a fingerprint component. The module boundary is binary-stable so that the builder can load pre-compiled permutation modules (LoadLibrary/dlopen) and verify the ABI contract per module.

4.5 The Building-Block Axes

After the N-phase extension, the matrix comprises **fourteen main axes plus roughly thirty sub-axes**: (1) Page-Type, (2) Node-Type, (3) Traversal (split into 3.A search-algorithm, 3.B cache-memory, 3.M mapping), (4) ValueHandle, (5) Memory-Layout, (6) Allocator (split into 6.1 allocation, 6.2 reclamation, 6.3 NUMA, 6.4 huge-page, 6.5 free-list), (7) Prefetch, (8) Concurrency (8.1 pattern, 8.2 locking-mode), (9) ISA, (10) Measurement, (11) Telemetry, (12) Hardware-Strategy *(new)*, (13) Scheduling-Strategy *(new)*, and (14) Identity. Per stack level a `std::variant` enumerates all concretizations; the Cartesian product yields more than 10^{11} permutations (Section **??**).

4.6 PRT-ART as Probe

The `PrtArtSearchEngineAdapter` implements the `SearchEngine` ABI by *composition* rather than direct inheritance, so the hybrid `PrtArtSearchEngine<Ts...>` API stays untouched; the

adapter pattern enables a compile-time fallback onto cache-engine building blocks (`resolve_baustein.hpp`) where the probe does not override an axis. PRT-ART contributes its own implementations in the page, layout, free-list, prefetch, concurrency-pattern, and measurement axes (4+2 pool allocator, distance-estimator prefetch, OLC with reserved value blocks, and the H1/H2/H3 metrics); for the remaining axes it reuses existing SOTA building blocks via permutation configuration.

4.7 CacheEngineBuilder and Three-Stage Evaluation

The builder orchestrates a three-stage evaluation: **stage 1** (cache-engine only) uses the default variant per axis; **stage 2** (probe only) replaces the overridden axes with the probe variants and keeps cache-engine defaults elsewhere; **stage 3** (full join, non-redundant) combines the default and all probe variants. Each stage emits thousands of non-redundant permutation binaries, each loaded at run time as an ABI-stable C++23 module. Stages map directly onto the three mandatory measurement series of Chapter **??**.

5 Implementation

This chapter describes the implementation thematically (not as a chronological development log): the three-repository split, the SOTA and allocator adapters, the permutation code generation and flag system, the concept/CRTP realization of the axes, concurrency and reclamation, the cache-coherence-aware telemetry, and the measurement pipeline.

5.1 Three-Repository Architecture

The implementation is distributed over three Git repositories with clearly separated responsibilities:

- `comdare-cache-engine` — **how** things are measured: the tool library and the builder. Key components: the builder (XML config parser, `generate_module_from_profile` code generation, permutation loop, module loader, and the seven-phase `ExperimentDriver`); the ABI headers (`search_engine`, `execution_engine`, `baustein_variants`, `resolve_baustein`); 30 SOTA profiles under `algorithm_profiles/sota`; and the test suites. The source tree follows a Pitchfork-style layout (`apps/` for executables, `libs/` for libraries, with `libs/common` for cross-cutting code).

- `comdare-prt-art` — the **probe**: the hybrid `PrtArtSearchEngine` (vector/map/tuple API), its ABI adapters, its building blocks (4+2 allocator pools, OLC with reserved blocks, multi-level layout, path-oriented prefetch, density tracker, hypothesis metrics H1/H2/H3), and the probe-side reimplementations.

- `probst-Diplomarbeit-cache-engine` — **what** is tested: the `messung_driver`, the experiment configurations, the evaluation pipeline, and the manuscript.

5.2 Adapters for State-of-the-Art Algorithms and Allocators

Each SOTA design and each allocator is integrated through a thin adapter that exposes the original implementation behind the engine ABI. The adapters follow a uniform pattern: a `COMDARE_HAVE_<X>` compile flag guards an `#if defined` block that wires the original API, with a safe fallback (e.g. `std::malloc` or a typed exception) when the original is unavailable. This keeps the build self-contained while allowing the original sources to be activated per dependency. The hierarchical-prefetch integration (P27) additionally ships a build-time call-graph analyzer (`p27_bundle_finder`, a C++23 port of the original `hp_soft` tool) that identifies functions whose subtree footprint exceeds a threshold as bundle entry points.

5.3 Permutation Code Generation and Flag System

The builder turns each profile into a pre-compiled C++23 module (`generate_module_from_profile`). Every permutation is identified by a bit-packed *permutation flag* structure. With the N-phase refinement of the matrix, the flag system grows from 9 banks ($\sim$50 bit) to 14 banks with sub-bank bitfields and a total **82-bit permutation identifier**, mirroring the fourteen axes and their sub-axes. A constraint filter masks out invalid axis combinations before any module is generated.

5.4 Concept/CRTP Realization of the Axes

Each axis is a topic directory containing a two-layer concept (a broad topic concept and a narrow axis concept) and a CRTP base class secured by a `requires` clause. The probe-versus-default selection per axis is resolved at compile time via `resolve_baustein.hpp` (`std::conditional_t` over tag specializations), so the hot path contains neither `virtual` calls nor run-time switches — the zero-cost abstraction of Section **??**.

5.5 Concurrency and Reclamation

Concurrency is itself an axis (8.1 pattern, 8.2 locking mode), realized through a concurrency manager that coordinates the disciplines OLC, HTM, STM, lock-free, RCU, and hazard pointers. Reclamation is a sub-axis of the allocator (6.2): epoch-based, RCU, hazard-pointer, and QSBR strategies are selectable per permutation. PRT-ART contributes OLC with reserved value blocks as its concurrency variant.

5.6 Telemetry against Cache-Coherence Effects

Per-node counters cause cache-line ping-pong on multi-core systems, which distorts the very measurements they collect. Following the Kuehn line of work (P28), the telemetry axis therefore offers cache-coherence-aware strategies: *leaf-only counters* (the main variant), *sampling* (every N-th leaf access), and *offline recompute* (bottom-up aggregation before reordering); the classic per-node counter is retained only as a labeled anti-pattern for comparison.

5.7 Measurement Pipeline

The evaluation tool chain is a sequence of small programs: `sample_data_generator` (deterministic test data without real hardware) $\rightarrow$ `messung_driver` (drives the series) $\rightarrow$ `binary_to_csv` (deserializes the binary measurement records) $\rightarrow$ `csv_to_latex` (booktabs tables with per-profile algorithm fact sheets) $\rightarrow$ `diagram_generator` (A4-aware TikZ plots) $\rightarrow$ `latex_to_pdf`. The measurement path in the execution engine is compiled in only under `-DCOMDARE_EXPERIMENT_MODE=ON`, so the production path carries no measurement overhead. Continuous integration (GitLab CI plus a mirrored GitHub workflow) runs across all three repositories.

6 Evaluation Methodology

This chapter describes how measurements are taken; the concrete results follow in Chapter **??**.

6.1 Three Measurement Series

The architecture prescribes three measurement series, all executed by the `messung_driver` through the `ExperimentDriver` library, configured either as three fixed series or via an external `messreihen.xml`.

Series A — PRT-ART vs. the state of the art. Compares the probe against the documented SOTA algorithms. Two modes: *A_defined* (quick verification against the 8 Tier-1 profiles) and *A_full* (all 30 SOTA profiles; the default, per the directive to measure as completely as possible).

Series B — cache-engine permutations. The pure "state of the art" baseline: all SOTA profiles without the PRT-ART probe.

Series C — merge of old and new. Concrete merge points between PRT-ART building blocks and cache-engine permutations, e.g. PRT-ART OLC + tcmalloc + ART page; PRT-ART 4+2 pool + HOT page; PRT-ART path prefetch + B^2-tree page; PRT-ART multi-level layout + Wormhole page.

6.2 ExperimentDriver Phases

The `ExperimentDriver` runs seven phases per series: (1) **enumerate** (parse the XML configs and build the permutation list); (2) **codegen** (one `comdare_perm_<fp>.cpp` per permutation, plus `generate_module_from_profile` for the SOTA profiles); (3) **compile**; (4) **load** (LoadLibrary/dlopen of all modules); (5) **execute** (`create_instance` + profile-aware `run_workload`); (6) **measure** (aggregate the binary measurement records); (7) **persist** (`measurements.csv` + `.json`). Two opt-in phases hot-compile missing modules and verify the ABI contract per module.

6.3 Hypotheses

H1 (page-type cost) The mean access cost per page-type variant differs systematically by workload: dense page types are expected to win under Zipfian (YCSB-A) and range-optimized pages under range scans (YCSB-E).

H2 (code quality per source) The code-quality score inherited from the original source repository correlates measurably with achieved throughput.

H3 (inline vs. external vs. chain distribution) The observed distribution of the three value-handle variants correlates with page density: dense pages favour inline, sparse pages external, and PRT-ART heuristics chain-ref.

6.4 Workloads and Datasets

Phase 5 routes the YCSB workload per permutation from the profile's traversal tag (Table **??**); modules without a profile use the global default workload.

Table 6.1: Profile-aware workload routing

Traversal tag	**YCSB workload**	**Characteristic**
`RANGE_SCAN` / `RANGE_HOT_PATH`	YCSB-E	range queries
`HOT_PATH` / `PRTART_HOT_PATH`	YCSB-A	50/50 read/update, Zipfian
`STANDARD` (default)	YCSB-C	read-only

6.5 Experimental Platforms

Measurements target a local workstation for development verification and the TU Dresden ZIH cluster (Barnard CPU, Capella GPU) for the full sweeps, plus the Tier-3 platforms (Ryzen 9 9950X3D, Intel i9 14900KS, and an HBM platform). An `IPlatformProbe` (phase 1) reports, per platform, the runnable subset of axes (e.g. AVX-512 only where available, HBM-aware only on HBM hardware).

6.6 Permutation Explosion and Reduction

With the N-phase extension from 11 to 14 axes plus ∼30 sub-axes, the space grows from ∼5.5 billion to more than 10^{11} permutations. Three mechanisms tame the explosion: (1) the `<expected_workload>` profile filter routes only compatible workloads; (2) `MessreihenMode::Defined` runs only the declared tuples (∼100–1000 per series) that feed the manuscript; (3) `Full` (and the more realistic `Full-Sampled`, e.g. every 1000th permutation) runs only on the ZIH cluster, respecting axis compatibility reported by the platform probe.

7 Results and Evaluation

This chapter presents the measurement results. Concrete diagrams and tables are generated from the measurement runs and included from the (language-neutral) `tikz/` and `tabellen/` directories; see the placeholders below. At the time of writing, the methodology and the evaluation pipeline are in place and verified on sample data; the real-hardware runs are pending (Section **??**).

7.1 Evaluation Pipeline

Per series run, the `messung_driver` writes `Code/_runs/<date>/<spec_id>/` with `measurements.csv` (one row per permutation: cycles, L1/L2/L3 misses, branches, throughput), `measurements.json`, and optional binary records. The evaluation uses `binary_to_csv` (deserialize and enrich with profile id, paper ref, axes), `csv_to_latex` (booktabs tables with per-profile fact sheets), and `diagram_generator` (A4-aware TikZ bar charts, scatter plots, heatmaps). The manuscript consumes these via `\input{tikz/<spec_id>/...}` and `\input{tabellen/<spec_id>...}`.

7.2 Series A: PRT-ART vs. State of the Art

The hypotheses H1–H3 (Section **??**) are evaluated here once the measurement data is available.

7.3 Series B: Cache-Engine Permutations

7.4 Series C: Full Join

7.5 Axis Sensitivity Analysis

Using the per-permutation records, this section quantifies which axis contributes most to the performance variance — the basis for recommending default configurations per workload class.

7.6 Discussion

The results are discussed against the leitmotif of the thesis: the transition from heuristic to informed cache management, i.e. whether telemetry-driven, automatically adapted configurations outperform statically chosen ones.

8 Conclusion and Outlook

8.1 Answering the Research Questions

RQ1 (Composition). The fourteen permutation axes (page, node, traversal, value-handle, memory-layout, allocator, prefetch, concurrency, ISA, measurement, telemetry, hardware-strategy, scheduling-strategy, identity) span a configuration space of more than 10^{11} permutations. State-of-the-art designs are reconstructed unambiguously through algorithm-profile XMLs: 30 SOTA profiles (8 Tier-1 + 22 Tier-2/3) demonstrate that the axis decomposition carries P01–P32. The variadic API specialization (1/2/N parameters onto `std::vector`/`std::map`/tuple) makes both algorithm- and container-oriented patterns expressible on one engine.

RQ2 (Measurement methodology). Measurement is split into a production mode (`COMDARE_EXPERIMENT_MODE` no overhead) and an experiment mode (the result aggregator is an integral execution-engine member). Profile-aware workload routing selects the workload per permutation from the traversal tag, ensuring fair comparison between range-query- and point-lookup-oriented designs.

RQ3 (Probe comparison). The concrete comparison of PRT-ART against the 30 SOTA profiles is pending the first measurement runs (Section **??**). The architecture is fully prepared: the PRT-ART profile, the three mandatory series templates, the `ExperimentDriver` with auto-pickup of the SOTA profiles, and the ABI-conformant adapter with all `std::map` operations and notify hooks.

8.2 Limitations

- **Module bodies are currently stubs:** the code-generation pipeline produces ABI-conformant skeleton modules per profile; the actual per-permutation algorithm body remains for a follow-up phase.
- **Hardware validation pending:** configuration is green locally, but the full `ctest` and end-to-end `messung_driver` runs across the BlockAO platforms (AVX2/AVX-512, ARM NEON/SVE) are outstanding.
- **Results chapter:** currently methodological; concrete measurements follow.

8.3 Outlook

In the short term: the end-to-end run of the three mandatory series on real hardware, the generation of the measurement diagrams, and the completion of the per-permutation module bodies. In the medium to long term: a GPU adapter (Grace Hopper cluster, ZIH), PIM-allocator integration (A16), and formal

verification of selected permutation classes (StarMalloc-style, A13). The three-repository architecture lets future students add further probe algorithms analogously to PRT-ART without modifying the cache-engine tool library — a key design principle for the scalability of the research infrastructure.

List of Figures

List of Tables

A Complete Measurement Series

B Code Structure

C Glossary

D Building-Block Matrix

E Architecture Decisions

Acknowledgments

Placeholder acknowledgments.